

Document #: P1901R2

Date: 2023-03-29

Reply-to: Daryl Haresign <cpp@daryl.haresign.com>

Audience: Library Working Group

Enabling the Use of `weak_ptr` as Keys in Unordered Associative Containers

Contents

- [Abstract](#)
- [History](#)
- [Motivation](#)
- [Proposal](#)
- [Alternatives Considered](#)
- [Wording](#)
- [References](#)
- [Acknowledgements](#)

Abstract

This paper proposes the addition of ownership based hashing and equality, to enable the use of `weak_ptr` as keys in unordered associative containers.

History

- R2:
 - Updated wording after LWG review
 - Added feature test macro
- R1:
 - Updated wording after LEWG review:
 - Removed non-transparent class template specializations
 - Added `is_transparent` to `owner_hash`
- R0:
 - Initial version.

Motivation

I recently found myself wanting to store a collection of `weak_ptr` objects. I neither had a separate key nor any requirements on the ordering of the objects. It seemed that using an `unordered_set` would be a good fit. It quickly became apparent, however, that it's simply not possible; there is no way to generate a stable hash or compare `weak_ptr` objects without support from the Standard Library.

Also relevant is the discussion in LWG Issue 1406, which labelled this as not a defect, and recommended a paper be brought forward to propose the addition.

Proposal

owner_hash and owner_equal

The obvious, and in my opinion best, option is to follow in the footsteps of `owner_less`, which can be used to enable `weak_ptr` keys in ordered associative containers.

N2637 was the paper through which `owner_less` was introduced into the standard.

The following methods and class templates would be added:

- `size_t shared_ptr::owner_hash() const noexcept;`
- `bool shared_ptr::owner_equal(...) const noexcept;`
- `size_t weak_ptr::owner_hash() const noexcept;`
- `bool weak_ptr::owner_equal(...) const noexcept;`
- `template <class T> struct owner_hash;`
- `template <class T> struct owner_equal;`

Note: LEWG review noted that the non-transparent versions were not required, so `owner_hash` and `owner_equal` did not need to be class templates. This part of the document has not been updated to reflect that change so as to keep the original proposal text intact.

`shared_ptr` and `weak_ptr` would gain two new methods each: `owner_hash` and `owner_equal`. `owner_hash` would return a ownership based hash and `owner_equal` would compare ownership.

In addition, two new class templates would be provided: `owner_hash` and `owner_equal`.

This option has the benefit of providing all the tools necessary for having a `weak_ptr` key in an unordered associative container.

The addition of the methods and template specializations for `shared_ptr` enables the ability to look up values in an unordered associative container from either the `weak_ptr` or the corresponding `shared_ptr`. It also enables storing `shared_ptr` keys based on the ownership equivalence relation, if such a thing is desired.

I don't believe this proposal exposes any more implementation details than are already exposed through `owner_less`.

To satisfy the *Cpp17Hash* requirements that the probability of a collision be low, a conforming implementation could simply hash the address of the control block, per `hash<void*>`.

Alternatives Considered

owner_ptr / owner / owner_id

This option would instead add the following methods:

- `const unspecified shared_ptr::owner_ptr() const noexcept;`
- `const unspecified weak_ptr::owner_ptr() const noexcept;`

The *unspecified* return type would need to be something for which there is, or could be, a `std::hash` specialization and an equality comparator (for example, a pointer to the control block, a guid stored in the control block, etc.).

This option would look to provide the bare minimum required, leaving it up to the user to define `owner_hash` and `owner_equal` in terms of this method.

shared_ptr_key / weak_ptr_key

This option would provide wrapper classes, `shared_ptr_key` and `weak_ptr_key`, which would have a `std::hash` specialization and an equality comparator. They would presumably be friends of `shared_ptr` and `weak_ptr` respectively.

Do Nothing

Whilst writing this paper, I discovered that it is actually possible for end users to do this right now in terms of `owner_before`, it just wouldn't be efficient.

`owner_hash` would be implemented to return a constant value — completely going against the *Cpp17Hash* requirements which state that the probability of a collision be low.

and

`owner_equal(p, q)` would be implemented to return `!p.owner_before(q) && !q.owner_before(p)`.

Wording

Feature-test macro

Add a feature test macro in the appropriate place [`version.sym`], respecting alphabetical order, with the value as the date of adoption.

```
#define __cpp_lib_smart_pointer_owner_equality_202XXXL // also in <memory>
```

Header `<memory>` synopsis [`memory.syn`]

In the header synopsis for `<memory>`, add `owner_hash` and `owner_equal`.

```
// [util.smartptr.ownerless], class template owner_less
template<class T = void> struct owner_less;

// [util.smartptr.owner.hash], struct owner_hash
struct owner_hash;

// [util.smartptr.owner.equal], struct owner_equal
struct owner_equal;
```

Class template `owner_less` [`util.smartptr.ownerless`]

Update the note as follows:

- [Note: Note that
 - `operator()` defines a strict weak ordering as defined in [`alg.sorting`];
 - ~~two `shared_ptr` or `weak_ptr` instances are equivalent under the equivalence relation defined by `operator()`, `!operator()(a, b) &&`~~

~~operator()(b, a), if and only if they share ownership or are both empty.~~

— !operator()(a, b) && !operator()(b, a) is true if and only if
a.owner_equal(b) is true.

— end note]

Class owner_hash [util.smartptr.owner.hash]

Add this new section under [util.smartptr].

Class owner_hash [util.smartptr.owner.hash]

The class owner_hash provides ownership-based hashing.

```
namespace std {  
    struct owner_hash {  
        template <class T>  
            size_t operator()(const shared_ptr<T>&) const noexcept;  
  
        template <class T>  
            size_t operator()(const weak_ptr<T>&) const noexcept;  
  
        using is_transparent = unspecified;  
    };  
}
```

```
template <class T>  
    size_t operator()(const shared_ptr<T>& x) const noexcept;  
template <class T>  
    size_t operator()(const weak_ptr<T>& x) const noexcept;
```

- Returns: x.owner_hash().
- [Note: For any object y where x.owner_equal(y) is true, x.owner_hash().
== y.owner_hash() is true. — end note]

Class owner_equal [util.smartptr.owner.equal]

Add this new section under [util.smartptr].

Class owner_equal [util.smartptr.owner.equal]

The class owner_equal provides ownership-based mixed equality comparisons of
shared and weak pointers.

```
namespace std {  
    struct owner_equal {  
        template <class T, class U>  
            bool operator()(const shared_ptr<T>&, const shared_ptr<U>&) const noexcept;  
        template <class T, class U>  
            bool operator()(const shared_ptr<T>&, const weak_ptr<U>&) const noexcept;  
        template <class T, class U>  
            bool operator()(const weak_ptr<T>&, const shared_ptr<U>&) const noexcept;  
        template <class T, class U>  
            bool operator()(const weak_ptr<T>&, const weak_ptr<U>&) const noexcept;  
  
        using is_transparent = unspecified;  
    };  
}
```

```

template <class T, class U>
bool operator()(const shared_ptr<T>& x, const shared_ptr<U>& y) const noexcept;
template <class T, class U>
bool operator()(const shared_ptr<T>& x, const weak_ptr<U>& y) const noexcept;
template <class T, class U>
bool operator()(const weak_ptr<T>& x, const shared_ptr<U>& y) const noexcept;
template <class T, class U>
bool operator()(const weak_ptr<T>& x, const weak_ptr<U>& y) const noexcept;

```

- Returns: `x.owner_equal(y)`.
- [Note: `x.owner_equal(y)` is true if and only if they share ownership or are both empty. — end note]

Class template `shared_ptr` [util.smartptr.shared]

Add to the observers section:

```

template<class U>
bool owner_before(const shared_ptr<U>& b) const noexcept;
template<class U>
bool owner_before(const weak_ptr<U>& b) const noexcept;
size_t owner_hash() const noexcept;
template<class U>
bool owner_equal(const shared_ptr<U>& b) const noexcept;
template<class U>
bool owner_equal(const weak_ptr<U>& b) const noexcept;

```

Observers [util.smartptr.shared.obs]

Update the `owner_before` specification as follows:

- Returns: An unspecified value such that
— ~~`x.owner_before(y)`~~ `owner_before(b)` defines a strict weak ordering as defined in [alg.sorting];
— ~~under the equivalence relation defined by `owner_before`;~~
~~`!a.owner_before(b) && !b.owner_before(a)`, two `shared_ptr` or `weak_ptr` instances are equivalent if and only if they share ownership or are both empty;~~
— `!owner_before(b) && !b.owner_before(*this)` is true if and only if `owner_equal(b)` is true.

Add after `owner_before`:

```

size_t owner_hash() const noexcept;

```

- Returns: An unspecified value such that, for any object `x` where `owner_equal(x)` is true, `owner_hash() == x.owner_hash()` is true.

```

template<class U>
bool owner_equal(const shared_ptr<U>& b) const noexcept;
template<class U>
bool owner_equal(const weak_ptr<U>& b) const noexcept;

```

- Returns: true if and only if `*this` and `b` share ownership or are both empty. Otherwise returns false.

- Remarks: owner_equal is an equivalence relation.

Class template weak_ptr [util.smartptr.weak]

Add to the observers section:

```
template<class U>
    bool owner_before(const shared_ptr<U>& b) const noexcept;
template<class U>
    bool owner_before(const weak_ptr<U>& b) const noexcept;
size_t owner_hash() const noexcept;
template<class U>
    bool owner_equal(const shared_ptr<U>& b) const noexcept;
template<class U>
    bool owner_equal(const weak_ptr<U>& b) const noexcept;
```

Observers [util.smartptr.weak.obs]

Update the owner_before specification as follows:

- *Returns:* An unspecified value such that
 - ~~x.owner_before(y)~~ owner_before(b) defines a strict weak ordering as defined in [alg.sorting];
 - ~~under the equivalence relation defined by owner_before,~~
~~!a.owner_before(b) && !b.owner_before(a), two shared_ptr or~~
~~weak_ptr instances are equivalent if and only if they share ownership or are~~
~~both empty.~~
 - !owner_before(b) && !b.owner_before(*this) is true if and only if
owner_equal(b) is true.

Add after owner_before:

```
size_t owner_hash() const noexcept;

- Returns: An unspecified value such that, for any object x where  

owner_equal(x) is true, owner_hash(.) == x.owner_hash(.) is true.

template<class U>  

    bool owner_equal(const shared_ptr<U>& b) const noexcept;  

template<class U>  

    bool owner_equal(const weak_ptr<U>& b) const noexcept;

- Returns: true if and only if *this and b share ownership or are both empty.  

Otherwise returns false.
- Remarks: owner_equal is an equivalence relation.

```

Class template enable_shared_from_this [util.smartptr.enab]

Update sample code as follows:

```
assert(!p.owner_before(q) && !q.owner_before(p) p.owner_equal(q)); // p and q share ownership
```

References

- LWG Issue 1406 — <http://www.open-std.org/jtc1/sc22/wg21/docs/lwg-closed.html#1406>
- N2637 Revisiting `std::shared_ptr` comparison — <https://wg21.link/n2637>
- *Cpp17Hash* requirements — [tab:cpp17.hash] in <https://wg21.link/n4830>

Acknowledgements

Thanks to Alexander Jones, Alisdair Meredith and Masud Rahman for their help reviewing, suggesting alternatives, and providing background on the history of `owner_less`.